2.2.19 Advanced Topic: How Evaluation Works

Evaluate the head of the expression.

Evaluate each element in turn.

Reorder, thread over lists, or flatten, if the function requires it.

Apply any rules you have specified.

Apply any built-in rules.

Evaluate the result.

The standard evaluation procedure for *Mathematica* expressions.

Mathematica goes through essentially the same evaluation procedure for any expression. Here is a simple example, where we assume that `a = 7`.

`2 a x + a^2 + 1`	here is the original expression
`Plus[Times[2, a, x], Power[a, 2], 1]`	this is the internal form
`Times[2, a, x]`	this is evaluated first
`Times[2, 7, x]`	`a` is evaluated to give 7
`Times[14, x]`	rules for `Times` give this result
`Power[a, 2]`	this is evaluated next
`Power[7, 2]`	here is the result after evaluating `a`
`49`	rules for `Power` give this result
`Plus[Times[14, x], 49, 1]`	here is the result after the arguments of `Plus` have been evaluated
`Plus[50, Times[14, x]]`	rules for `Plus` give this result
`50 + 14 x`	the result is printed like this

A simple example of evaluation in *Mathematica*.

You can watch some of the evaluation process in *Mathematica* by switching on "tracing" using the command `On[ ]`.

First set `a` to 7.

```
In[1]:= a = 7
Out[1]= 7
```

Now switch on tracing.

```
In[2]:= On[ ]
```

This shows the stages in the evaluation procedure.

```
In[3]:= 2 a x + a^2 + 1
a::trace: a --> 7.
Times::trace: 2 7 x --> 14 x.
a::trace: a --> 7.
                  2
Power::trace: 7  --> 49.
Plus::trace: 14 x + 49 + 1 --> 1 + 49 + 14 x.
Plus::trace: 1 + 49 + 14 x --> 50 + 14 x.
Out[3]= 50 + 14 x
```

This switches off trace messages.

```
In[4]:= Off[ ]
```

The Order of Rule Application

Mathematica stores rules associated with a particular object in a definite order. What order the rules are in will depend on the order in which you entered them, and on what reorderings were made by `Set`. You can find out what order the rules for a particular object are in by typing `?`*f*. (If *f* has a usage message, you will have to type `??`*f* to see the actual rules defined for *f*.)

When *Mathematica* evaluates an expression, it tries each of the rules associated with the expression in turn, and returns the result from the first rule that applies.

With an expression like *f*`[`*g*`[`*x*`]]`, however, there are two possible sets of rules that can apply: those associated with *f*, and those associated with *g*. In a case like this, *Mathematica* follows the principle of *first* trying rules associated with *g*, and then, only if none of these apply, trying rules associated with *f*.

The general goal is to apply specific transformation rules before general ones. By applying rules associated with the arguments of a function before rules associated with the function itself, *Mathematica* allows you to give rules for special arguments that override the general rules for evaluation of the function with any arguments.

This defines a rule for `f[g[x_]]`, to be associated with `f`.

```
In[1]:= f/: f[g[x_]] := frule[x]
```

This defines a rule for `f[g[x_]]`, associated with `g`.

```
In[2]:= g/: f[g[x_]] := grule[x]
```

The rule associated with `g` is tried before the rule associated with `f`.

```
In[3]:= f[g[2]]
Out[3]= grule[2]
```

If you remove rules associated with `g`, the rule associated with `f` is used.

```
In[4]:= Clear[g] ;  f[g[1]]
Out[4]= frule[1]
```

> Rules associated with g are applied before rules associated with f in the expression f`[`g`[`x`]]`.

The order in which rules are applied.

Mathematica usually applies any built-in transformation rules for a particular object *after* it applies rules that you have explicitly given. *Mathematica* therefore evaluates an expression like f`[`g`[x]]` in four stages:

- Apply rules you have given associated with g;
- Apply built-in rules associated with g;
- Apply rules you have given associated with f;
- Apply built-in rules associated with f.

The fact that *Mathematica* applies rules associated with g before those associated with f in an expression like f`[`g`[`x`]]` is important for a number of purposes.

Given a rather generic operation such as composition, you may want to define special cases for particular kinds of objects. You will, however, usually also want to give general rules, in this example for the composition operation, to be used if none of the special cases apply.

Here is a definition associated with `q` for composition of "q objects".

```
In[5]:= q/: comp[q[x_], q[y_]] := qcomp[x, y]
```

Here is a general rule for composition, associated with `comp`.

```
In[6]:= comp[f_[x_], f_[y_]] := gencomp[f, x, y]
```

If you compose two q objects, the rule associated with q is used.

```
In[7]:= comp[q[1], q[2]]
Out[7]= qcomp[1, 2]
```

If you compose r objects, the general rule associated with comp is used.

```
In[8]:= comp[r[1], r[2]]
Out[8]= gencomp[r, 1, 2]
```

Non-Standard Argument Evaluation

The standard *Mathematica* scheme for evaluating a function is first to evaluate the arguments to the function, and then to evaluate the function itself. There are some functions, however, that do not use this standard scheme. Instead, they either do not evaluate their arguments at all, or evaluate them in a special way that they themselves control.

Assignment functions are one example. If you type `a = 1`, the `a` on the left-hand side is not evaluated before the assignment is performed. You can see that there would be trouble if the `a` were to be evaluated. For example, you might previously have made the assignment `a = 7`. Then, if the `a` in `a = 1` was evaluated, you would get the nonsensical expression `7 = 1`.

Control structures are another important example of functions that evaluate their arguments in non-standard ways. For example, `Do[`*expr*`, {`*n*`}]` maintains *expr* in an unevaluated form, so that it can explicitly evaluate *expr* *n* separate times.

`Hold` is the prototypical example of a function that does not evaluate its arguments.

```
In[1]:= Hold[1 + 1]
Out[1]= Hold[1 + 1]
```

You can use hold to maintain an expression in unevaluated form, then evaluate it using `Release`.

```
In[2]:= Release[%]
Out[2]= 2
```

Functions that use non-standard argument evaluation carry attributes like `HoldAll`, which effectively specify that certain arguments are to be treated as if they were enclosed in `Hold` functions, and therefore to be left unevaluated. You can override the implicit `Hold` by giving function arguments of the form `Release[`*arg*`]`.

f`[Release[`*arg*`]]`	evaluate *arg* immediately, even though the attributes of *f* specify that it should be held

Overriding non-standard argument evaluation.

The presence of the Release overrides the non-standard argument evaluation usually used by Hold.

```
In[3]:= Hold[Release[1 + 1]]
Out[3]= Hold[2]
```

This assigns a to have value b. The left-hand side of an assignment like this is not usually evaluated.

```
In[4]:= a = b
Out[4]= b
```

The assignment a = 7 would simply reset a to 7. Release[a] evaluates to b, however, before the assignment is done.

```
In[5]:= Release[a] = 7
Out[5]= 7
```

As a result, it is b, rather than a, that is set to 7.

```
In[6]:= b
Out[6]= 7
```

Evaluation in Assignments

symbol = *value*	*symbol* is not evaluated; *value* is evaluated
symbol := *value*	neither *symbol* nor *value* are evaluated
f[*args*] = *value*	*args* are evaluated; left-hand side as a whole is not
f[Literal[*arg*]] = *value*	*f*[*arg*] is assigned, without evaluating *arg*
Release[*lhs*] = *value*	left-hand side is evaluated completely

Evaluation in assignments.

When you type in something like f[1 + 1] = 2, *Mathematica* first evaluates the arguments of f to get f[2] = 2, and only then performs the assignment. As a result, *Mathematica* stores a transformation rule for f[2], rather than for f[1+1]. A transformation rule for f[1+1] would not be particularly useful, since the expression f[1+1] could never actually arise in the process of evaluation. Assuming that f does not have any attributes such as HoldAll, f[1+1] will always be evaluated to f[2] before any transformation rules are tried.

There are some cases, however, where it is necessary to make assignments with the argument of the left-hand side, or some part of it, not evaluated. You can do this by wrapping the function Literal around the parts that are not to be evaluated. Literal is a HoldFirst function, which maintains its argument in an unevaluated form. However, *Mathematica* replaces Literal[*expr*] by *expr* in the actual expression that is assigned, but without evaluating *expr*.

`Integrate` assumes here that `y_` is a constant, and so gives the peculiar result `x_ y_` for the integral.

```
In[1]:= Integrate[y_, x_]
Out[1]= (x_) (y_)
```

Mathematica leaves integrals like this in a form that matches the pattern `Integrate[y_, x_]`.

```
In[2]:= Integrate[q[x], x]
Out[2]= Integrate[q[x], x]
```

This defines a value for `f` when its argument is any object of the form `Integrate[y_, x_]`. The `Literal` prevents `Integrate[y_, x_]` from being evaluated at the time of assignment.

```
In[3]:= f[Literal[Integrate[y_, x_]]] := fint[y, x]
```

The `Literal` has been stripped out of the expression, but `Integrate[y_, x_]` has been left unevaluated.

```
In[4]:= ?f
f
f/: f[Integrate[y_, x_]] := fint[y, x]
```

The pattern now matches in this case.

```
In[4]:= f[Integrate[q[x], x]]
Out[4]= fint[q[x], x]
```

Iteration Functions

Iteration functions such as `Table` and `Sum`, as well as `Plot` and `Plot3D`, evaluate their arguments in a slightly complicated way.

First, the "iterator specification" that appears as the second argument of an expression like `Table[`*f*`, {`*i*`, `*imax*`}]` is evaluated. The object *i* cannot have a value at this point. If it does have a value, `Table` prints a message, and stops.

The expression *f* to be tabulated is maintained in an unevaluated form. The evaluation of `Table` involves assigning a succession of values to the iteration variable *i*, and in each case evaluating *f*. Finally, the value assigned to *i* is removed.

The fact that `Random[ ]` is evaluated four separate times means that you get four different pseudorandom numbers.

```
In[1]:= Table[Random[ ], {4}]
Out[1]= {0.353297, 0.0205034, 0.117374, 0.546051}
```

The `Sum` in this case is evaluated separately for each value of *x*.

```
In[2]:= Table[Sum[x^i, {i, 8}], {x, 4}]
Out[2]= {8, 510, 9840, 87380}
```

Using `Release`, you can make the `Sum` be evaluated first.

```
In[3]:= Table[Release[Sum[x^i, {i, 8}]], {x, 4}]
Out[3]= {8, 510, 9840, 87380}
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

This defines fac to give the factorial when it has an integer argument, and to give NaN otherwise.

```
In[4]:= fac[n_Integer] := n! ;  fac[x_] := NaN
```

In this form, fac[i] is not evaluated until an explicit integer value has been assigned to i.

```
In[5]:= Table[fac[i], {i, 5}]
Out[5]= {1, 2, 6, 24, 120}
```

Using Release forces fac[i] to be evaluated with i left as a symbolic object.

```
In[6]:= Table[Release[fac[i]], {i, 5}]
Out[6]= {NaN, NaN, NaN, NaN, NaN}
```

Infinite Recursion

The general principle that *Mathematica* follows in evaluating expressions is to go on applying transformation rules until the expressions no longer change. This means, for example, that if you make an assignment like `x = x + 1`, *Mathematica* should go into an infinite loop. In fact, *Mathematica* stops after a definite number of steps, determined by the value of the global variable `$RecursionLimit`. You can always stop *Mathematica* earlier by explicitly interrupting it.

This assignment could cause an infinite loop. *Mathematica* stops after a number of steps determined by $RecursionLimit.

```
In[1]:= x = x + 1
General::recursion: Recursion depth of 256 exceeded.
Out[1]= 254 + Hold[1 + x]
```

When *Mathematica* stops without finishing evaluation, it returns a held result. You can continue the evaluation by explicitly calling Release.

```
In[2]:= Release[%]
General::recursion: Recursion depth of 256 exceeded.
Out[2]= 507 + Hold[1 + x]
```

An assignment like `x = x + 1` is obviously circular. When you set up more complicated recursive definitions, however, it can be much more difficult to be sure that the recursion terminates, and that you will not end up in an infinite loop. The main thing to check is that the right-hand sides of your transformation rules will always be different from the left-hand sides. This ensures that evaluation will always "make progress", and *Mathematica* will not simply end up applying the same transformation rule to the same expression over and over again.

Some of the trickiest cases occur when you have rules that depend on complicated `/;` conditions (see page 279). One particularly awkward case is when the condition involves a "global variable". *Mathematica* may think that the evaluation is finished because the expression did not change. However, a side effect of some other operation could change the value of the global variable, and so should lead to a new result in the evaluation. The best way to avoid this kind of difficulty is not to use global variables in `/;` conditions. If all else fails, you can type `Update[s]` to tell

Mathematica to update all expressions involving s. `Update[ ]` tells *Mathematica* to update absolutely all expressions.